Documentación del desarrollo de una aplicación de GPS

Pablo Tuñón Laguna Jorge Vančo Sampedro

Índice

Introducción 4
Funciones de Grafo.py 5
Clase arista 5
Clase grafo 6
Inicializador 6
Es dirigido 6
Agregar un vértice 7
Agregar una arista 7
Eliminar vértice 8
Eliminar arista 8
Obtener arista 9
Obtener aristas 9
Lista vértices 9
Lista de adyacencia 10
Grado saliente 10
Grado entrante 11
Grado 11
Algoritmo de Dijkstra 12
Camino mínimo 13
Algoritmo de Prim 13
Algoritmo de Kruskal 14
Convertir a networkx 15
Callejero.py 16
Info velocidad 16
Clase cruce 16
Clase calle 17
Functions.py 19
Get vector 19
Get tercera coordenada producto vectorial 19
Giro 19
Producto vectorial 20
Norma vectorial 20

Coseno de ángulo entre dos vectores 20
Formateo ordinal 20
Obtención de direcciones 21
Obtención de caminos mínimos 22
Crear mapa 23
Calles que cortan una calle 24
Números de portales por calles 24
Distancia euclídea 24
Encontrar cruce 24
Implementación del GPS 26
Create_graph.py 26
Comparar y 26
Colocar grupos 26
Definir grupos 27
Agrupar puntos 27
Crear cruces 27
Sort cruces calle 27
Distance 27
Get aristas 27
Unir cruce 27
Create graphs 27
Aplicación 29

Introducción

GPS corresponde con las siglas “Global Positioning System” y tal y como su nombre indica es un dispositivo capaz de localizar posiciones y por tanto trayectorias, distancias… Para que se puedan conocer distancias y posiciones se requiere de un sistema de referencia, ese sistema de referencia suele ser un conjunto de puntos mapeados a la sección a referenciar. Dichos puntos son accesibles desde otros puntos del mapa generando así las distintas trayectorias para alcanzar un destino (un punto) desde un origen (otro punto).

La definición tomada en el párrafo anterior es muy similar a la definición de grafo, concretamente a la de grafo no dirigido:

” Diagrama que representa mediante puntos y líneas las relaciones entre pares de elementos”[1]

El conjunto de puntos son los puntos mapeados de referencia mientras que las relaciones son las trayectorias posibles entre puntos. Es por ello que para el desarrollo de la aplicación de GPS se van a tomar puntos de referencia y relaciones entre dichos puntos con el fin de generar dicho sistema de posicionamiento. Sin embargo, cabe destacar que no se poseen datos para mapear todo el globo terráqueo por lo que el desarrollo de la aplicación se va a limitar a la capital española: Madrid.

Comprendido ya que se necesita un mapa de puntos y relaciones entre las mismas para crear un grafo y poder realizar funciones de posicionamiento sobre el mismo, se necesita definir qué se va a tomar como punto y qué se va a tomar como relación de puntos. Se tiene un conjunto de datos que proporcionan latitud y longitud de los distintos portales de Madrid, así como las direcciones de las calles en las que se encuentran y los cruces de dichas calles con otras calles del conjunto de datos. Es por ello que los puntos o más técnicamente vértices del grafo serán los números mientras que las “relaciones” mejor conocidas como aristas (en teoría de grafos) serán las distintas calles de Madrid, dado que no se puede acceder a un portal atravesando un edificio. Aparte esto permitirá no sólo hacer posicionamientos y cálculos teóricos sino cálculos reales, ya que se puede usar como gps en la ciudad de Madrid.

Para el funcionamiento de la aplicación se han desarrollado 7 ficheros y se ha usado un octavo, que limpia los datos de los datasets, los cuales proporcionan la información en la forma de csvs.

[1] DRAE, https://dle.rae.es/grafo

Funciones de Grafo.py

En este fichero sólo se encuentran contenidas dos clases, arista y grafo. En ella se realizan algunas implementaciones que aunque no se usan para la creación de la app de gps, sirven para comprender mejor la teoría de grafos.

Clase arista

Esta clase sirve para recoger los objetos de tipo arista que posteriormente se usarán en la creación de un grafo. Contiene los atributos: u(nodo origen), v(nodo destino, en un grafo no dirigido estos coinciden), data(información adicional de la arista, en el caso del gps el nombre de la calle) y weight(en caso de construir un grafo pesado, el "coste" de tomar dicha arista).

Aparte cuenta con 7 métodos (sin contar el de inicialización) estos permiten:

- Representar como una string el contenido de la arista.
- Comparar el origen, final y datos adicionales de dos aristas.
- Escoger entre dos aristas la que tiene el mismo o menos peso.
- Escoger entre dos aristas la que tiene el mismo o más peso.
- Escoger entre dos aristas la que más peso tiene.
- Escoger entre dos aristas la que menos peso tiene
- Genera un valor hash para el objeto, útil en posteriores funciones.

```python
class Arista:
    def __init__(self, s, t, data, weight) -> None:
        self.u = s
        self.v = t
        self.data = data
        self.weight = weight

    def __repr__(self) -> str:
        return f"({self.u}, {self.v}, w: {self.weight}), data:{self.data}"

    def __eq__(self, other) -> bool:
        return self.u == other.u and self.v == other.v and self.data == other.data

    def __le__(self, other) -> bool:
        return self.weight <= other.weight

    def __ge__(self, other) -> bool:
        return self.weight >= other.weight

    def __gt__(self, other) -> bool:
        return self.weight > other.weight

    def __lt__(self, other) -> bool:
        return self.weight < other.weight

    def __hash__(self) -> int:
        return hash(f"({self.u}, {self.v}, {self.data}, {self.weight})")
```

Figura 1: clase arista, autoría propia.

Clase grafo

Almacena grafos dirigidos y no dirigidos (el tipo que posteriormente se empleará para el desarrollo del gps). Un grafo dirigido es aquél en el que las aristas tienen un vértice inicial y uno final y sólo se pueden recorrer desde el inicial hasta el final, nunca en sentido inverso, mientras que en un grafo no dirigido, no hay vértice origen y final, sino que en tanto que haya una arista conectando dos vértices esta se puede recorrer, similar a tener una arista dirigida doble por cada arista del grafo.

La clase también proporciona funciones que permiten transformar el objeto grafo en un grafo de networkx librería de Python que facilita el trabajo con grafos. Así mismo también contiene algoritmos de recorrido de grafos así como otras herramientas para su análisis y comprensión.

Inicializador

Inicializa el objeto grafo con un valor booleano que indica si es un grafo dirigido o si por el contrario, no lo es. También crea un diccionario que representa la adyacencia del grafo, las claves serán los nodos del grafo por lo que se puede obtener la adyacencia y el contenido de una arista del grafo en cualquier momento.

```python
def __init__(self, dirigido: bool = False) -> None:
    """Crea un grafo dirigido o no dirigido.

    Args:
        dirigido (bool): Flag que indica si el grafo es dirigido (verdadero) o no (falso).

    Returns:
        Grafo o grafo dirigido (según lo indicado por el flag)
        inicializado sin vértices ni aristas.
    """

    # Flag que indica si el grafo es dirigido o no.
    self.dirigido: bool = dirigido

    """
    Diccionario que almacena la lista de adyacencia del grafo.
    adyacencia[u]:  diccionario cuyas claves son la adyacencia de u
    adyacencia[u][v]:   Contenido de la arista (u,v), es decir, par (a,w) formado
                        por el objeto almacenado en la arista "a" (object) y su peso "w" (float).
    """
    self.adyacencia: Dict[object, Dict[object, Tuple[object, float]]] = {}
```

Figura 2: inicializador de la clase grafo, autoría propia.

Es dirigido

Sencillamente verifica si el grafo es dirigido, devolviendo True si el grafo es dirigido y False en caso contrario.

```python
def es_dirigido(self) -> bool:
    """Indica si el grafo es dirigido o no

    Args: None
    Returns: True si el grafo es dirigido, False si no.
    Raises: None
    """
    return self.dirigido
```

Figura 3: verificación de si el grafo es dirigido.

Agregar un vértice

Agrega un vértice al grafo introduciendo el hash del objeto que representa el vértice en el diccionario de adyacencia y creando en su interior un nuevo diccionario.

```python
def agregar_vertice(self, v: object) -> None:
    """Agrega el vértice v al grafo.

    Args:
        v (object): vértice que se quiere agregar. Debe ser "hashable".
    Returns: None
    Raises:
        TypeError: Si el objeto no es "hashable".
    """
    try:
        h: int = hash(v)
    except TypeError:
        raise TypeError(f"El objeto {v} no es hasheable")

    self.adyacencia[v] = dict()
```

Figura 4: agregar un vértice al grafo, autoría propia.

Agregar una arista

Si existen el hash de los vértices s y t, lo consigue (para poder modificar posteriormente el diccionario de adyacencia), en caso de que no exista devuelve TypeError. Verificado que los vértices son hasheables, se verifica que ambos están en el grafo(ya que si no no se podría crear la arista) y se crea una arista que va del nodo s al nodo t con un peso introducido al llamar la función. Posteriormente, se introduce dicha arista en el diccionario correspondiente a la s en como valor de la t, es decir la arista que une s con t, en caso de que le grafo sea no dirigido, también se introduce la arista con el peso en el diccionario correspondiente a la t en la posición s. Es decir, tal y como se ha explicado anteriormente, si todas las aristas dirigidas de un grafo son dobles, el grafo no es dirigido.

```python
def agregar_arista(
    self, s: object, t: object, data: object = None, weight: float = 1
) -> None:
    """Si los objetos s y t son vértices del grafo, agrega
    una arista al grafo que va desde el vértice s hasta el vértice t
    y le asocia los datos "data" y el peso weight.
    En caso contrario, no hace nada.

    Args:
        s (object): vértice de origen (source).
        t (object): vértice de destino (target).
        data (object, opcional): datos de la arista. Por defecto, None.
        weight (float, opcional): peso de la arista. Por defecto, 1.
    Returns: None
    Raises:
        TypeError: Si s o t no son "hashable".
    """

    try:
        hash_s: int = hash(s)
        hash_t: int = hash(t)

    except TypeError as e:
        raise TypeError(e)

    if s in self.adyacencia and t in self.adyacencia:
        # Se crea el objeto arista
        arista = Arista(s, t, data, weight)

        # Se añade la arista s -> t
        self.adyacencia[s][t] = (arista, weight)

        # Si no es un grafo dirigido, tambien se añade la arista t -> s
        if not self.es_dirigido():
            self.adyacencia[t][s] = (arista, weight)
```

Figura 5: agregar una arista al grafo, autoría propia.

Eliminar vértice

Esta función permite eliminar un vértice del grafo, independientemente de si es o no dirigido. En primer lugar se verifica que el vértice es posible que esté en el grafo, verificando si es hasheable, si no, devolvería TypeError. Posteriormente verifica si el vértice se encuentra en el diccionario de adyacencia y elimina toda la información contenida bajo ese vértice, a continuación se usa un bucle for para eliminar toda aparición del vértice en las adyacencias de los demás vértices del grafo.

```python
def eliminar_vertice(self, v: object) -> None:
    """Si el objeto v es un vértice del grafo lo elimina.
    Si no, no hace nada.

    Args:
        v (object): vértice que se quiere eliminar.
    Returns: None
    Raises:
        TypeError: Si v no es "hashable".
    """

    try:
        hash_v: int = hash(v)
    except TypeError as e:
        raise TypeError(e)

    if v in self.adyacencia:
        # Elimina v de la adyacencia
        self.adyacencia.pop(v)

        # Elimina las aristas que van hasta el vertice v
        for vertice in self.adyacencia:
            if v in self.adyacencia[vertice]:
                self.adyacencia[vertice].pop(v)
```

Figura 6: eliminación de un vértice de un grafo, autoría propia.

Eliminar arista

Esta función recibe los vértices que conecta la arista, tras verificar que estos se pueden encontrar en el grafo(realizando su hash y devolviendoo TypeError en caso de que no estén), se verifica si ambos nodos se encuentran en el grafo. En dicho caso, se busca la posición s en la lista de adyacencia y se elimina toda la información de t, en caso de que sea un grafo no dirigido se realiza el proceso inverso, acabando con toda la información de s en la posición t del diccionario de adyacencia.

```python
def eliminar_arista(self, s: object, t: object) -> None:
    """Si los objetos s y t son vértices del grafo y existe
    una arista de u a v la elimina.
    Si no, no hace nada.

    Args:
        s: vértice de origen de la arista (source).
        t: vértice de destino de la arista (target).
    Returns: None
    Raises:
        TypeError: Si s o t no son "hashable".
    """
    try:
        hash_s: int = hash(s)
        hash_t: int = hash(t)

    except TypeError as e:
        raise TypeError(e)

    if s in self.adyacencia and t in self.adyacencia:
        adyacencia_s: Dict[object, Tuple[object, float]] = self.adyacencia[s]

        # Elimina la arista s -> t de la adyacencia de s
        if t in adyacencia_s:
            adyacencia_s.pop(t)

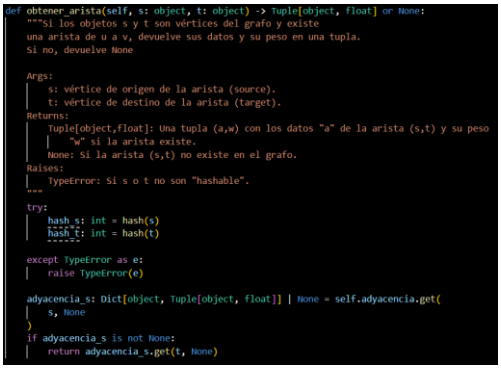
        # Si no es dirigido, se borra también la arista t -> s
        if not self.es_dirigido():
            adyacencia_t: Dict[object, Tuple[object, float]] = self.adyacencia[t]

            # Elimina la arista t -> s de la adyacencia de t
            if s in adyacencia_t:
                adyacencia_t.pop(s)
```

Figura 7: Eliminar una arista de un grafo, autoría propia.

Obtener arista

La función a explicar trata de devolver la arista que un el vértice s con el vértice t. En primer lugar realiza el hash de las funciones con el objetivo de comprobar si pueden o no estar presentes en el grafo devolviendo TypeError en caso de que no sea posible que se encuentren en el grafo. A continuación, devuelve la arista que une s con t en caso de que esta exista. Es decir, en el diccionario de adyacencia busca los valores de clave s y en ellos busca los valores de la clave t, que es una tupla que contiene un objeto arista y su peso.



```
def obtener_arista(self, s: object, t: object) -> Tuple[object, float] or None:
    """Si los objetos s y t son vértices del grafo y existe
    una arista de u a v, devuelve sus datos y su peso en una tupla.
    Si no, devuelve None

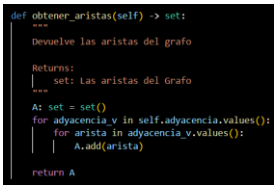
    Args:
        s: vértice de origen de la arista (source).
        t: vértice de destino de la arista (target).
    Returns:
        Tuple[object,float]: Una tupla (a,w) con los datos "a" de la arista (s,t) y su peso
            "w" si la arista existe.
        None: Si la arista (s,t) no existe en el grafo.
    Raises:
        TypeError: Si s o t no son "hashable".
    """
    try:
        hash_s: int = hash(s)
        hash_t: int = hash(t)
        """"""
    except TypeError as e:
        raise TypeError(e)

    adyacencia_s: Dict[object, Tuple[object, float]] | None = self.adyacencia.get(
        s, None
    )
    if adyacencia_s is not None:
        return adyacencia_s.get(t, None)
```

Figura 8: devuelve la arista que une dos nodos en caso de que esta exista, autoría propia.

Obtener aristas

Devuelve todas las aristas del grafo a través de dos bucles for anidados. Crea un set de aristas y posteriormente recorre todos los valores de todas las claves del diccionario de adyacencia tomando así todas las aristas.



```
def obtener_aristas(self) -> set:
    """
    Devuelve las aristas del grafo

    Returns:
        set: Las aristas del Grafo
    """
    A: set = set()
    for adyacencia_v in self.adyacencia.values():
        for arista in adyacencia_v.values():
            A.add(arista)

    return A
```

Figura 9: obtiene todas las aristas de un grafo y las devuelve en forma de set, autoría propia.

Lista vértices

Devuelve una lista con todos los vértices de un grafo, esto lo realiza convirtiendo en una lista todas las claves del diccionario de adyacencia, ya que hay una por cada nodo.



```
def lista_vertices(self) -> List[object]:
    """Devuelve una lista con los vértices del grafo.

    Args: None
    Returns:
        List[object]: Una lista [v1,v2,...,vn] de los vértices del grafo.
    Raises: None
    """
    return list(self.adyacencia.keys())
```

Figura 10: devuelve una lista con todos los vértices del grafo, autoría propia.

Lista de adyacencia

La lista de adyacencia es una lista en la que se muestran todos los nodos a los que se puede llegar desde el nodo en cuestión, en el caso de un grafo no dirigido, todos los nodos que están unidos por una arista al que se calcula la adyacencia. En el caso de uno dirigido, todos los nodos a los que se puede llegar partiendo del vértice del que se calcula la adyacencia. La función en cuestión verifica que el nodo puede estar en el grafo, a continuación verifica que el nodo se encuentra en el diccionario de adyacencia y si es así, devuelve todas las claves del diccionario que contiene, ya que estas son los nodos presentes en la adyacencia del vértice a calcular la lista de adyacencia.

```python
def lista_adyacencia(self, u: object) -> List[object] or None:
    """Si el objeto u es un vértice del grafo, devuelve
    su lista de adyacencia, es decir, una lista [v1,...,vn] con los vértices
    tales que (u,v1), (u,v2),..., (u,vn) son aristas del grafo.
    Si no, devuelve None.

    Args: u vértice del grafo
    Returns:
        List[object]: Una lista [v1,v2,...,vn] de los vértices del grafo
            adyacentes a u si u es un vértice del grafo
        None: si u no es un vértice del grafo
    Raises:
        TypeError: Si u no es "hashable".
    """
    try:
        hash_u: int = hash(u)

    except TypeError as e:
        raise TypeError(e)

    adyacencia_u: Dict[object, Tuple[object, float]] | None = self.adyacencia.get(
        u, None
    )

    if adyacencia_u is not None:
        adyacencia_u = list(adyacencia_u.keys())

    return adyacencia_u
```

Figura 11: cálculo de la lista de adyacencia de un nodo, autoría propia.

Grado saliente

Después de verificar que el nodo es hasheable y que por tanto puede estar en el grafo, se obtiene la adyacencia de el nodo a calcular el grado y al calcular la longitud del diccionario que contiene se obtiene el grado saliente, ya que son el número de vértices que salen del nodo, es decir, el número de aristas cuyo origen es el nodo a calcular el grado saliente, independientemente de si es dirigido o no dirigido.

```python
def grado_saliente(self, v: object) -> int or None:
    """Si el objeto v es un vértice del grafo, devuelve
    su grado saliente, es decir, el número de aristas que parten de v.
    Si no, devuelve None.

    Args:
        v (object): vértice del grafo
    Returns:
        int: El grado saliente de u si el vértice existe
        None: Si el vértice no existe.
    Raises:
        TypeError: Si u no es "hashable".
    """
    try:
        hash_v: int = hash(v)

    except TypeError as e:
        raise TypeError(e)

    adyacencia_v: Dict[object, Tuple[object, float]] | None = self.adyacencia.get(
        v, None
    )

    if adyacencia_v is None:
        return None

    return len(adyacencia_v)
```

Figura 12: cálculo del grado saliente de un nodo del grafo, autoría propia.

Grado entrante

El grado entrante en un grafo no dirigido es idéntico al saliente ya que las aristas son dobles, de entrada y de salida, sin embargo, en el caso de un grafo dirigido el grado entrante es el número de aristas que llegan al vértice.

En la función después de verificar que el vértice puede estar en el grafo, en caso de que el vértice no esté en el grafo se devuelve none, si no es dirigido se devuelve la adyacencia y si es dirigido, se recorre cada lista de adyacencia de cada uno de los vértices y se contabilizan el número de vértices que llegan al vértice a calcular el grado entrante.

```python
def grado_entrante(self, v: object) -> int or None:
    """Si el objeto v es un vértice del grafo, devuelve
    su grado entrante, es decir, el número de aristas que llegan a v.
    Si no, devuelve None.

    Args:
        v (object): vértice del grafo
    Returns:
        int: El grado entrante de u si el vértice existe
        None: Si el vértice no existe.
    Raises:
        TypeError: Si v no es "hashable".
    """
    try:
        hash_v: int = hash(v)
    except TypeError as e:
        raise TypeError(e)

    if v not in self.adyacencia:
        return None

    if not self.es_dirigido():
        return self.grado_saliente(v)

    grado = 0
    for adyacencia_u in self.adyacencia.values():
        if v in adyacencia_u:
            grado += 1

    return grado
```

Figura 13: cálculo del grado entrante de un vértice en función de si es o no dirigido, autoría propia.

Grado

En un grafo dirigido el grado de un vértice es idéntico a su grado saliente mientras que en un grafo no dirigido el grado es tanto el grado saliente como el entrante ya que estos son idénticos. Por lo tanto, después de comprobar que el vértice es hasheable y que este se encuentra en el grafo(en caso contrario la función devolvería None), devuelve el grado saliente del vértice.

```python
def grado(self, v: object) -> int or None:
    """Si el objeto v es un vértice del grafo, devuelve
    su grado si el grafo no es dirigido y su grado saliente si
    es dirigido.
    Si no pertenece al grafo, devuelve None.

    Args:
        v (object): vértice del grafo
    Returns:
        int: El grado grado o grado saliente de u según corresponda
            si el vértice existe
        None: Si el vértice no existe.
    Raises:
        TypeError: Si v no es "hashable".
    """
    try:
        hash_v: int = hash(v)
    except TypeError as e:
        raise TypeError(e)

    if v not in self.adyacencia:
        return None

    return self.grado_saliente(v)
```

Figura 14: cálculo del grado de un vértice como el grado saliente del mismo, autoría propia.

Algoritmo de Dijkstra

Nacido en 1930 en Rotterdam,Holanda, Edsger Wybe Dijkstra, fue un gran físico, matemático e investigador de nuevas técnicas de computación. En 1956 propuso su algoritmo de caminos mínimos en el cual partiendo de un nodo "origen" calcula los caminos con el menor peso posible a cada uno de los nodos, es este algoritmo el que se encuentra recogido en la función.

Después de verificar que el nodo puede estar en el grafo ya que es hasheable y que además se encuentra en el grafo. A continuación, crea 3 diccionarios:

- padre: encargado de anotar los padres de cada nodo en el camino más corto desde el nodo origen, inicializados todos como None.
- visitado: encargado de anotar si un nodo ha sido o no visitado mediante un valor booleano inicializado como False en todos los vértices del nodo(y por tanto claves del diccionario).
- d: mantiene un registro de la distancia acumulada más corta desde el origen hasta cada nodo del grafo, inicializada como infinito salvo la distancia del nodo origen a sí mismo que es 0.

Posteriormente el código usa una cola de prioridad (heapq) para gestionar los nodos no visitados y las distancias acumulativas. Hasta que la cola de prioridad esté vacía, se realiza un bucle que hace los siguientes procesos:

- Extracción de nodo con distancia acumulativa más corta.
- Si no se había visitado, se marca como visitado en el diccionario correspondiente, y se añaden a la cola de prioridad los nodos adyacentes.
- En caso de que se encuentre un camino más corto, se actualizan las distancias y los nodos padres.

```python
def dijkstra(self, origen: object) -> Dict[object, object]:
    """Calcula un Árbol de Caminos Mínimos para el grafo partiendo
    del vértice "origen" usando el algoritmo de Dijkstra. Calcula únicamente
    el árbol de la componente conexa que contiene a "origen".

    Args:
        origen (object): vértice del grafo de origen
    Returns:
        Dict[object,object]: Devuelve un diccionario que indica, para cada vértice alcanzable
            desde "origen", qué vértice es su padre en el árbol de caminos mínimos.
    Raises:
        TypeError: Si origen no es "hashable".
    Example:
        Si G.dijksra(1)={2:1, 3:2, 4:1} entonces 1 es padre de 2 y de 4 y 2 es padre de 3.
        En particular, un camino mínimo desde 1 hasta 3 sería 1->2->3.
    """
    try:
        hash_origen: int = hash(origen)

    except TypeError as e:
        raise TypeError(e)

    if origen not in self.adyacencia:
        return None

    padre = {}
    visitado = {}
    d = {}

    for v in self.adyacencia:
        padre[v] = None
        visitado[v] = False
        d[v] = INFTY

    d[origen] = 0
    Q = []

    heapq.heappush(Q, (d[origen], origen))

    while Q:
        _, nodo = heapq.heappop(Q)

        if visitado[nodo] == False:
            visitado[nodo] = True
            for x, arista in self.adyacencia[nodo].items():
                a, w = arista

                if d[x] > d[nodo] + w:
                    d[x] = d[nodo] + w
                    padre[x] = nodo
                    heapq.heappush(Q, (d[x], x))

    return padre
```

Figura 15: uso del algoritmo de Dijkstra para obtener el diccionario que marca para cada nodo cuál es su nodo padre de tal forma que se pueda encontrar un árbol de caminos mínimos desde el origen, autoría propia.

Camino mínimo

Se busca encontrar el camino mínimo entre dos nodos, es una aplicación directa del algoritmo de Dijkstra. En primer lugar, se verifica que ambos nodos pueden estar en el grafo y que además se encuentran en el mismo. Posteriormente se recorre de forma recursiva el árbol iniciando en el destino y finalizando cuando se llega al origen, finalmente se devuelve el camino recursivo invertido de tal forma que el primer nodo de la lista sea el origen y el último el destino.

```python
def camino_minimo(self, origen: object, destino: object) -> List[object]:
    """Calcula el camino mínimo desde el vértice origen hasta el vértice
    destino utilizando el algoritmo de Dijkstra.

    Args:
        origen (object): vértice del grafo de origen
        destino (object): vértice del grafo de destino
    Returns:
        List[object]: Devuelve una lista con los vértices del grafo por los que pasa
            el camino más corto entre el origen y el destino. El primer elemento de
            la lista es origen y el último destino.
    Example:
        Si G.dijksra(1,4)=[1,5,2,4] entonces el camino más corto en G entre 1 y 4 es 1->5->2->4.
    Raises:
        TypeError: Si origen o destino no son "hashable".
    """

    try:
        hash_origen: int = hash(origen)
        hash_destino: int = hash(destino)

    except TypeError as e:
        raise TypeError(e)

    if destino not in self.adyacencia or origen not in self.adyacencia:
        return None

    arbol_min = self.dijkstra(origen)
    camino = []
    buscando_camino = True

    while buscando_camino:
        camino.append(destino)
        destino = arbol_min[destino]

        if destino is None:
            buscando_camino = False
        elif destino == origen:
            camino.append(origen)
            buscando_camino = False

    return camino[::-1]
```

Figura 16: cálculo del camino mínimo desde un nodo hasta otro haciendo uso del algoritmo de Dijkstra, autoría propia.

Algoritmo de Prim

En 1930 fue descubierto por el matemático checo Vojtech Jarnik, posteriormente, en 1957 fue descubierto por Robert Prim, matemático estadounidense y finalmente Dijkstra lo redescubrió en 1959. El algoritmo parte de un vértice inicial al cual se anexan siempre las aristas de nodos adyacentes con menor peso, evitando siempre que se repitan los vértices, generando así un árbol abarcador mínimo.

En esta implementación no se pide un vértice para inicializar la construcción del árbol de caminos mínimos, ya que no es necesaria, se toma el primer nodo de la lista de prioridad. Lo primero que realiza la función es inicializar una cola de prioridad y dos diccionarios:

- Q: cola de prioridad que contiene tuplas de vértices y de pesos mínimos.
- padre: que mantiene un registro de los padres de los vértices del árbol, inicializando estos como None.

- coste_mínimo: almacena los costes mínimos para llegar a cada vértice, inicializando estos a infinito.

A continuación, se recorren vértices de la cola de prioridad hasta que esta se vacía, en cada iteración:

- Se extrae el vértice de coste mínimo de la cola.
- Se examinan los vértices no procesados.
- En caso de encontrar un costo menor para llegar a un vértice adyacente, se actualizan los valores de coste mínimo y padre.

```python
def prim(self) -> Dict[object, object]:
    """Calcula un Árbol Abarcador Mínimo para el grafo
    usando el algoritmo de Prim.

    Args: None
    Returns:
        Dict[object,object]: Devuelve un diccionario que indica, para cada vértice del
            grafo, qué vértice es su padre en el árbol abarcador mínimo.
    Raises: None
    Example:
        Si G.prim()={2:1, 3:2, 4:1} entonces en un árbol abarcador mínimo tenemos que:
            1 es padre de 2 y de 4
            2 es padre de 3
    """
    Q: list = []
    padre = dict()
    coste_minimo = dict()

    # Inicializamos variables
    for v in self.lista_vertices():
        padre[v] = None
        coste_minimo[v] = INFTY
        heapq.heappush(Q, (coste_minimo[v], v))

    # Se recorren todos los vértices
    while Q:
        _, v = heapq.heappop(Q)

        for x in set(self.adyacencia[v]) & set((x for _, x in Q)):
            _, w = self.obtener_arista(v, x)

            if w < coste_minimo[x]:
                indice: int = Q.index((coste_minimo[x], x))
                coste_minimo[x] = w
                padre[x] = v
                Q[indice] = (w, x)
                heapq.heapify(Q)

    return padre
```

Figura 17: algoritmo de prim para obtener árbol abarcador mínimo, autoría propia.

Algoritmo de Kruskal

El matemático y estadístico estadounidense Joseph B.Kruskal es el responsable de descubrir en 1956 este algoritmo, obteniendo como resultado un árbol abarcador mínimo. El algoritmo considera todas las aristas y toma siempre la de menor peso, una vez que la tiene, si no genera ciclos, la añade al árbol abarcador mínimo.

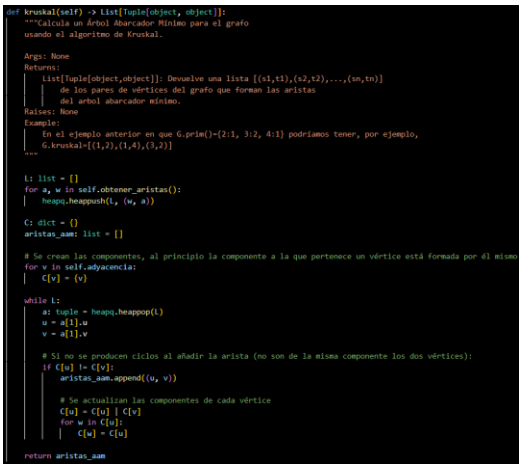
Inicialmente se crea una lista de prioridad, un diccionario y una lista corriente:

- L: lista de prioridad que almacena las aristas ordenadas por peso.

- C: componentes conexas, además se asigna a cada vértice una componente que inicialmente consiste en sólo ese vértice.
- aristas_aam: aristas pertenecientes al árbol abarcador mínimo

Posteriormente se itera hasta que se queda sin nodos la lista de prioridad:

- Se elimina el nodo de menor peso de L.
- Se divide en componentes.
- Si los componentes no son idénticos, no forman ciclos.
- Si no forman ciclos se añaden a aristas_aam, y se actualizan los componentes de cada vértice.



```
def kruskal(self) -> List[Tuple[object, object]]:
    """Calcula un Árbol Abarcador Mínimo para el grafo
    usando el algoritmo de Kruskal.

    Args: None
    Returns:
        list[Tuple[object,object]]: Devuelve una lista [(s1,t1),(s2,t2),...,(sn,tn)]
            de los pares de vértices del grafo que forman las aristas
            del arbol abarcador mínimo.
    Raises: None
    Example:
        En el ejemplo anterior en que G.prim()={2:1, 3:2, 4:1} podríamos tener, por ejemplo,
        G.kruskal=[(1,2),(1,4),(3,2)]
    """

    L: list = []
    for a, w in self.obtener_aristas():
        heapq.heappush(L, (w, a))

    C: dict = {}
    aristas_aam: list = []

    # Se crean las componentes, al principio la componente a la que pertenece un vértice está formada por él mismo
    for v in self.adyacencia:
        C[v] = {v}

    while L:
        a: tuple = heapq.heappop(L)
        u = a[1].u
        v = a[1].v

        # Si no se producen ciclos al añadir la arista (no son de la misma componente los dos vértices):
        if C[u] != C[v]:
            aristas_aam.append((u, v))

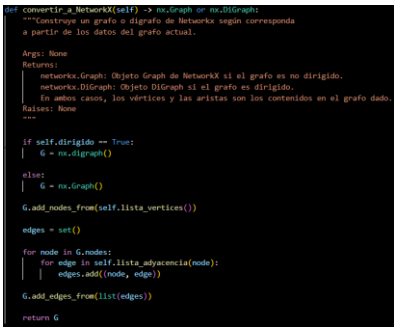
            # Se actualizan las componentes de cada vértice
            C[u] = C[u] | C[v]
            for w in C[u]:
                C[w] = C[u]

    return aristas_aam
```

Figura 18: obtención de árbol abarcador mínimo mediante el algoritmo de Kruskal.

Convertir a networkx

Se crea un grafo de networkx a partir de un objeto grafo. En caso de que sea dirigido o no se inicializa de una forma u otra. Posteriormente, haciendo uso de la lista de vértices se añaden todos los vértices, y finalmente se añaden todas las aristas obtenidas a través de un set que isa un doble bucle anidado para recorrer las listas de adyacencia de los nodos y mete en el set todas las aristas.



```
def convertir_a_NetworkX(self) -> nx.Graph or nx.DiGraph:
    """Construye un grafo o digrafo de Networkx según corresponda
    a partir de los datos del grafo actual.

    Args: None
    Returns:
        networkx.Graph: Objeto Graph de NetworkX si el grafo es no dirigido.
        networkx.DiGraph: Objeto DiGraph si el grafo es dirigido.
        En ambos casos, los vértices y las aristas son los contenidos en el grafo dado.
    Raises: None
    """

    if self.dirigido == True:
        G = nx.digraph()

    else:
        G = nx.Graph()

    G.add_nodes_from(self.lista_vertices())

    edges = set()

    for node in G.nodes:
        for edge in self.lista_adyacencia(node):
            edges.add((node, edge))

    G.add_edges_from(list(edges))

    return G
```

Figura 19: conversión de objeto grafo a objeto grafo de la librería networkx, autoría propia.

Callejero.py

En este fichero se encuentra contenido información sobre la velocidad (en la forma de una variable de tipo entero y un diccionario) además de una clase cruce y una clase calle. Este fichero junto con clean_dgt.py son los encargados de formatear los datos para la creación del grafo.

Info velocidad

Tal y como se ha comentado antes en este fichero una variable de tipo entero que marca la velocidad en calles estándar, definida como 50. El diccionario del fichero también hace referencia a velocidades, salvo que en este caso indica la velocidad por tipo de calle, siendo los tipos de calle las claves y las velocidades los valores.

```
VELOCIDADES_CALLES: Dict[str, int] = {
    "AUTOVIA": 100,
    "AVENIDA": 90,
    "CARRETERA": 70,
    "CALLEJON": 30,
    "CAMINO": 30,
    "ESTACION DE METRO": 20,
    "PASADIZO": 20,
    "PLAZUELA": 20,
    "COLONIA": 20,
}
VELOCIDAD_CALLES_ESTANDAR = 50
```

Figura 20: Velocidades al desplazarte por distintos tipos de calles.

Clase cruce

Aporta información sobre los cruces, para iniciar un objeto de este tipo se requieren los siguientes datos:

- Coordenadas x e y, almacenadas posteriormente en formato de tupla.
- Localización: una lista que contiene la latitud y la longitud del cruce.
- Calles: una lista que almacena el número de calles que intersecta.

Aparte del iniciador hay 8 funciones más que permiten operar los cruces a continuación se muestra lo que puede hacer cada una:

- Añadir una calle a la lista de calles.
- Añadir múltiples calles a la list de calles mediante un bucle for.
- Verifica si una calle se encuentra en la lista de calles de un cruce.
- Compara dos clases y devuelve True si las coordenadas de dos cruces son idénticas.
- Al comparar dos cruces, toma las coordenadas menores o iguales.
- Al comparar dos cruces, toma las coordenadas menores.

- Devuelve el hash de la tupla de coordenadas.
- Representa como una cadena las coordenadas del cruce y las calles que se cortan en el cruce.

```python
class Cruce:
    # Completar esta clase con los datos y métodos que se necesite asociar a cada cruce

    def __init__(
        self, coord_x: int, coord_y: int, location: list, points: list
    ) -> None:
        self.coord_x: int = coord_x
        self.coord_y: int = coord_y
        self.point = (coord_x, coord_y)
        self.location = location  # [latitud, longitud]
        self.points = points
        self.calles: list = []

    def add_calle(self, calle) -> None:
        self.calles.append(calle)

    def add_calles(self, calles) -> None:
        for calle in calles:
            self.add_calle(calle)

    def has_calle(self, calle) -> bool:
        if isinstance(calle, int):
            index = 1
        else:
            index = 0

        return calle in list(zip(*self.calles))[index]

    def __eq__(self, other) -> bool:
        if type(other) is type(self):
            return (self.coord_x == other.coord_x) and (self.coord_y == other.coord_y)
        else:
            return False

    def __le__(self, other) -> bool:
        return (self.coord_x, self.coord_y) <= (other.coord_x, other.coord_y)

    def __lt__(self, other) -> bool:
        return (self.coord_x, self.coord_y) < (other.coord_x, other.coord_y)

    def __hash__(self) -> int:
        return hash((self.coord_x, self.coord_y))

    def __repr__(self) -> str:
        return f"({self.coord_x}, {self.coord_y}) -> {self.calles}"
```

Figura 21: Clase cruce y todos sus métodos, autoría propia.

Clase calle

Esta clase contiene toda la información que se necesita para crear un objeto de tipo calle. En primer lugar se tiene el método inicializador que inicializa los atributos de la calle:

- Código de la calle de tipo entero.
- Nombre de la calle, tipo cadena.
- Los cruces de esa calle, es un set que contiene todos los cruces de la propia calle con otras calles (haciendo uso del objeto cruce).
- Clase de calle, para aplicar la velocidad correcta.
- Velocidad de la calle, se toma la velocidad correspondiente con el tipo de vía y se pasa a cm/min.

Aparte hay dos métodos más que permiten operar las calles:

- Una función permite añadir cruces a la calle.
- Otra permite devolver un diccionario con el código, el nombre y los cruces de una calle.

```python
class Calle:
    # velocidad
    # clase de la vía
    # codigo de número
    # codigo de via
    # nombre de la via
    def __init__(self, codigo, clase, nombre) -> None:
        self.codigo = int(codigo)
        self.nombre = nombre  # nombre
        self.cruces: set = set()
        self.clase = clase
        self.velocidad = (
            VELOCIDADES_CALLES.get(clase, VELOCIDAD_CALLES_ESTANDAR) * 5000 / 3
        )  # pasarlo a cm/min

    def add_cruce(self, cruce) -> None:
        self.cruces.add(cruce)

    def toDict(self) -> Dict[str, Any]:
        return {
            "codigo": self.codigo,
            "nombre": self.nombre,
            "cruces": self.cruces,
        }
```

Figura 22: Clase calle y sus métodos, autoría propia.

Functions.py

A lo largo de este fichero se definen funciones para realizar la posterior navegación en la red de calles de Madrid. Así, se proporcionan direcciones entre dos cruces de la ciudad, considerando diferentes aspectos como la longitud de las calles, el tiempo de viaje y la velocidad de conducción.

Get vector

Esta función calcula el vector entre dos cruces.

```python
def get_vector(cruce1, cruce2) -> Tuple:
    return (cruce2.coord_x - cruce1.coord_x, cruce2.coord_y - cruce1.coord_y)
```

Figura 23: obtención del vector de un cruce, autoría propia.

Get tercera coordenada producto vectorial

Calcula la tercera coordenada del producto vectorial de dos vectores dados bidimensionales.

```python
def get_tercera_coordenada_producto_vectorial(v1, v2) -> float:
    """
    Calcula el valor de la tercera coordenada del producto vectorial
    """
    x1, y1 = v1
    x2, y2 = v2
    return x1 * y2 - y1 * x2
```

Figura 24: obtención de la tercera coordenada del producto vectorial, autoría propia.

Giro

Determina el sentido de giro: si la tercera coordenada del producto vectorial obtenida a partir de la función anterior es positiva, se realizará un giro a la izquierda. En caso contrario, se trata de un giro a la derecha.

```python
def get_giro(v1, v2) -> str:
    """
    Decide hacia donde hay que girar
    """
    if get_tercera_coordenada_producto_vectorial(v1, v2) > 0:
        return "izquierda"
    else:
        return "derecha"
```

Figura 25: determinación del giro al llegar a un cruce, autoría propia.

Producto vectorial

Calcula el producto vectorial de dos vectores dados bidimensionales.

```python
def dot_product_vector(v1, v2) -> float:
    """
    Producto escalar de dos vectores
    """
    return v1[0] * v2[0] + v2[1] * v1[1]
```

Figura 26: producto vectorial de dos vectores, autoría propia.

Norma vectorial

Sirve para calcular la norma de un vector, es decir, el número no negativo que describe la extensión de este en el espacio.

```python
def vector_norm(v) -> float:
    """
    Norma del vector
    """
    return np.sqrt(np.square(v[0]) + np.square(v[1]))
```

Figura 27: norma de un vector, autoría propia.

Coseno de ángulo entre dos vectores

Calcula el coseno del ángulo entre dos vectores al dividir su producto escalar entre el producto de las normas de ambos vectores, es decir, $cos(\theta) = \frac{\vec{v_1} \cdot \vec{v_2}}{|\vec{v_1}| \cdot |\vec{v_2}|}$

```python
def cos_angle(v1, v2) -> float:
    """
    Coseno del ángulo que forman dos vectores
    """
    return dot_product_vector(v1, v2) / (vector_norm(v1) * vector_norm(v2))
```

Figura 28: coseno del ángulo entre dos vectores, autoría propia.

Formateo ordinal

Encuentra el carácter ordinal para un número, es decir, el número seguido de su respectivo indicador ordinal.

```python
def get_caracter_ordinal(num) -> str:
    """
    Encuentra el caracter ordinal para un número
    """
    caracteres_ordinales = {1: "er", 3: "er"}
    resto = num % 10
    return caracteres_ordinales.get(resto, "º")
```

Figura 29: obtención del carácter ordinal de un número, autoría propia.

Obtención de direcciones

Esta función tiene como objetivo generar direcciones detalladas para seguir un camino mínimo en un grafo. La función utiliza información del grafo proporcionado para obtener detalles sobre las aristas y calles asociadas a cada tramo del camino.

La variable direcciones se inicia como una lista vacía que almacenará las instrucciones generadas. Se identifica el cruce inicial como el primer nodo del camino. Se verifica si el camino mínimo tiene menos de 2 nodos; en tal caso, se retorna un mensaje indicando que no se ha encontrado el camino seleccionado.

Se obtiene la primera arista y su longitud asociada del grafo, correspondiente al tramo entre el cruce inicial y el siguiente nodo en el camino.

La longitud total del trayecto y el tiempo total se actualizan con la información del primer tramo.

Se itera sobre los nodos restantes en el camino. Para cada par de nodos consecutivos, se obtiene la arista correspondiente y se calculan vectores y otros datos relacionados con las aristas y calles involucradas.

Posteriormente se compara la calle de la arista actual con la calle de la arista anterior para decidir si se sigue recto o se realiza un giro (cuyo sentido quedará determinado por el signo del producto vectorial, como ya se ha comentado). Se verifica el ángulo entre los vectores de las aristas para determinar si el giro es lo suficientemente significativo como para ser mencionado.

Se generan direcciones y se acumulan tanto la longitud como el tiempo estimado totales, considerando la topología de las calles, giros y distancias.

```python
def get_direcciones(camino_minimo, grafo) -> List[str]:
    """
    Obtiene las direcciones para un camino
    """

    direcciones = []

    cruce_inicial = camino_minimo[0]

    if len(camino_minimo) < 2:
        return ["No se ha encontrado el camino seleccionado"]

    arista1, _ = grafo.obtener_arista(cruce_inicial, camino_minimo[1])
    longitud1 = arista1.data["longitud"]

    longitud_total_trayecto = longitud1

    tiempo_total = arista1.data["tiempo"]

    cruces_pasados = 0
    for i, cruce in enumerate(camino_minimo[2:], 1):
        arista2, _ = grafo.obtener_arista(camino_minimo[i], cruce)

        vector1 = get_vector(camino_minimo[i - 1], camino_minimo[i])
        calle1 = arista1.data["calle"]

        vector2 = get_vector(camino_minimo[i], cruce)
        longitud2 = arista2.data["longitud"]
        calle2 = arista2.data["calle"]

        if calle1 == calle2:
            longitud1 += longitud2
            cruces_pasados += 1

            if i == len(camino_minimo) - 2:
                caracter_ordinal = get_caracter_ordinal(cruces_pasados + 1)
                direcciones.append(
                    f"Continúe por <strong>{calle1}</strong> {(longitud1 + longitud2)/100:0.0f} metros, en el {cruces_pasados + 1}{caracter_ordinal} cruce está su destino."
                )
        else:
            # Se comprueba el ángulo por si el giro es demasiado pequeño como para decir girar
            if cos_angle(vector1, vector2) < 0.9:
                giro = get_giro(vector1, vector2)
                if (
                    cruces_pasados > 0
```

Figura 30: obtención de direcciones primera parte, autoría propia.



Figura 31: obtención de direcciones segunda parte, autoría propia

Obtención de caminos mínimos

Esta función tiene como objetivo encontrar los caminos mínimos y las direcciones de dos rutas entre dos cruces dados en un grafo. La función toma como entrada el nodo de salida de la ruta, el nodo de llegada, y un par de grafos que representan las distancias y las velocidades entre los cruces.

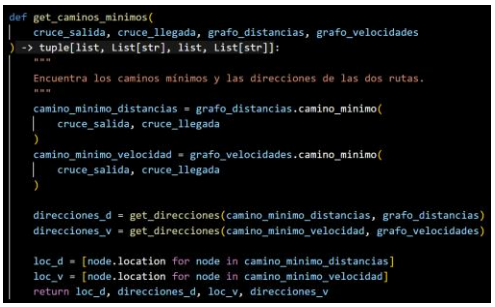
Se calculan los caminos mínimos entre el cruce de salida y el de llegada, que se almacenan en las variables camino_minimo_distancias y camino_minimo_velocidad.

Se generan las direcciones a partir de la función descrita anteriormente para los respectivos grafos.

Se crea una lista de ubicaciones (loc_d) que contiene las ubicaciones de los nodos en el camino mínimo basado en distancias.

De manera similar, se crea una lista de ubicaciones (loc_v) que contiene las ubicaciones de los nodos en el camino mínimo basado en velocidades.

Finalmente, la función devuelve una lista de ubicaciones para el camino mínimo basado en distancias, las direcciones detalladas para la ruta basada en distancias, la lista de ubicaciones para el camino mínimo basado en velocidades y las direcciones detalladas para la ruta basada en velocidades.



```
def get_caminos_minimos(
    cruce_salida, cruce_llegada, grafo_distancias, grafo_velocidades
) -> tuple[list, list[str], list, list[str]]:
    """
    Encuentra los caminos mínimos y las direcciones de las dos rutas.
    """
    camino_minimo_distancias = grafo_distancias.camino_minimo(
        cruce_salida, cruce_llegada
    )
    camino_minimo_velocidad = grafo_velocidades.camino_minimo(
        cruce_salida, cruce_llegada
    )

    direcciones_d = get_direcciones(camino_minimo_distancias, grafo_distancias)
    direcciones_v = get_direcciones(camino_minimo_velocidad, grafo_velocidades)

    loc_d = [node.location for node in camino_minimo_distancias]
    loc_v = [node.location for node in camino_minimo_velocidad]
    return loc_d, direcciones_d, loc_v, direcciones_v
```

Figura 32: Obtención de los caminos mínimos, autoría propia.

Crear mapa

Se encarga de crear un mapa interactivo utilizando la librería Folium de Python. Este mapa puede mostrar una ruta entre dos ubicaciones (salida y llegada) con caminos basados en distancias y velocidades.

Primero se crea un objeto de mapa de Folium (m) con una ubicación central inicial en [40.416775, -3.45] y un nivel de zoom inicial de 11.

Si se proporciona un cruce de salida, se añade un marcador verde en el mapa con un ícono de estrella.

Si se proporciona un cruce de llegada, se añade un marcador azul en el mapa con un ícono de bandera a cuadros.

Una vez obtenida la lista de ubicaciones para el camino basado en distancias, se agrega una polilínea verde al mapa que representa la ruta.

De manera similar, dada la lista de ubicaciones para el camino basado en velocidades, se agrega una polilínea azul al mapa que representa la ruta.

La función devuelve el objeto de mapa interactivo de Folium que permite visualizar y comparar ambas rutas.

```python
def create_map(
    cruce_salida=None, cruce_llegada=None, loc_d=None, loc_v=None
) -> folium.Map:
    """
    Crea el mapa de folium
    """
    m = folium.Map(
        location=[40.416775, -3.45],  # -3.703790],
        zoom_start=11,
        zoom_control=False,
    )

    # Añadir salida
    if cruce_salida is not None:
        folium.Marker(
            location=cruce_salida.location,
            icon=folium.Icon(color="green", icon="star"),
        ).add_to(m)

    # Añadir destino
    if cruce_llegada is not None:
        folium.Marker(
            location=cruce_llegada.location,
            icon=folium.Icon(color="blue", icon="fa-flag-checkered", prefix="fa"),
        ).add_to(m)

    # Añadir caminos
    if loc_d:
        folium.PolyLine(loc_d, color="green", weight=3, opacity=0.8).add_to(m)
    if loc_v:
        folium.PolyLine(loc_v, color="blue", weight=3, opacity=0.8).add_to(m)

    return m
```

Figura 33: crear mapa de puntos, autoría propia.

Calles que cortan una calle

Esta función realiza una búsqueda en los cruces asociados a una calle dada para obtener los nombres de las calles que intersecan con ella y devuelve una lista de estos nombres, excluyendo la calle de referencia.

```python
def get_calles_cortan_calle(calle, nombre_calle) -> List:
    """
    Consigue el nombre de las calles que cortan a dicha calle
    """
    new_calles = set()
    for cruce in calle.cruces:
        new_calles = new_calles | set(list(zip(*cruce.calles))[0])
    new_calles -= {nombre_calle}
    return list(new_calles)
```

Figura 34: obtención de calles que cortan otras calles autoría propia.

Números de portales por calles

Obtiene los números de portales de una calle específica a partir del dataframe de direcciones. Se filtra el dataframe para obtener las filas asociadas a la calle proporcionada, que se guardan en un diccionario donde los números de portal son las claves y las coordenadas son los valores.

```python
def get_numeros_portales_calle(nombre_calle, direcciones_df) -> None:
    """
    Consigue los números de portales de una calle
    """
    k = direcciones_df[direcciones_df["Nombre completo de la via"] == nombre_calle][
        ["Coordenada X (Guia Urbana) cm", "Coordenada Y (Guia Urbana) cm", "Numero"]
    ].apply(
        lambda r: (
            (r["Coordenada X (Guia Urbana) cm"], r["Coordenada Y (Guia Urbana) cm"]),
            r["Numero"],
        ),
        axis=1,
    )
    diccionario_codigos = {}

    for coord, num in k:
        diccionario_codigos[num] = coord
    return diccionario_codigos
```

Figura 35: Calcula los números de portales por calles, autoría propia.

Distancia euclídea

Esta función calcula la distancia euclidiana entre dos puntos en un plano bidimensional a partir de la fórmula siguiente: $\sqrt{(p_0 - q_0)^2 + (p_1 - q_1)^2}$

```python
def distance_points(p, q) -> float:
    """
    Calcula la distancia entre dos puntos
    """
    return np.sqrt((p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
```

Figura 36: cálculo de la distancia euclídea

Encontrar cruce

Esta función encuentra el cruce asociado a una calle dada, ya sea a partir de un número de portal específico o del nombre de otra calle que cruza.

Si el parámetro imputado es un número, se asume que es un número de portal y se utiliza para obtener las coordenadas asociadas a ese número de portal desde el diccionario numeros_portales.

Se inicializan un par de variables para almacenar el nodo más cercano y la distancia mínima encontrada hasta el momento. Se itera sobre los nodos de cruce asociados a la calle y se calcula la distancia entre las coordenadas del número de portal y las coordenadas del nodo de cruce. Si la distancia es menor que la mínima encontrada hasta el momento, se actualizan las variables con la información del nodo actual. Al finalizar la iteración se devuelve el nodo de cruce más cercano al número de portal.

En el caso en el que la calle seleccionada no es un número, se itera sobre los nodos de cruce asociados a la calle y se verifica si el nodo tiene la calle seleccionada como una de sus calles. Si se encuentra un nodo con esta condición, se devuelve ese nodo de cruce.

En ambos casos si se encuentra un cruce adecuado, se devuelve el nodo de cruce correspondiente.

```python
def find_cruce(calle, calle_seleccionada, numeros_portales) -> Any | None:
    """
    Encuentra el cruce elegido para una calle a partir del número de portal o el nombre de la calle que la cruza
    """
    # Si es un número se encuentra el cruce más cercano al portal
    if calle_seleccionada.isnumeric():
        numero_portal = int(calle_seleccionada)
        coord_numero_portal = numeros_portales.get(numero_portal, None)
        if coord_numero_portal is None:
            return

        min_node = None
        min_distance = None
        # Se busca el cruce más cercano
        for nodo in calle.cruces:
            d = distance_points(coord_numero_portal, nodo.point)
            if min_node is None or d < min_distance:
                min_node = nodo
                min_distance = d
        return min_node

    else:
        # Se busca el cruce con la calle elegida
        for nodo in calle.cruces:
            if nodo.has_calle(calle_seleccionada):
                return nodo
```

Figura 37: obtención del cruce, autoría propia.

Implementación del GPS

Con todas las funciones que permiten realizar operaciones matemáticas, trabajar con grafos y formatear las calles para un uso adecuado finalmente se va a implementar el GPS. Esta sección se va a dividir en los dos ficheros encargados de dicho proceso: créate_graph.py y gps.py.

Create_graph.py

Este fichero contiene 9 funciones necesarias para la creación y navegación por el gps.

Comparar y
La función toma dos coordenadas una base y otra la del cruce y las compara conforme un radio, esto servirá para hacer converger todos los puntos de un cruce en uno de tal manera que se pueda plantear un vértice por intersección y no uno por cada calle que llega a la intersección.

```
def compare_y(row, y, R) -> bool:
    y2 = row[columna_y]
    return y2 <= y + R and y2 >= y - R
```

Figura 38: Compara la y con un radio dado, autoría propia.

Colocar grupos
Es una función encargada de asignar grupos a elementos de un grafo. Recibe los siguientes parámetros:

- Cruces: dataframe con los nodos del grafo.
- Cruce: nodo específico del grafo.
- Fila del dataframe de cruces que corresponde al nodeo cruce.
- Y es la abcisa de la referencia.
- R es el valor que muestra la distancia máxima permitida.
- Aparte recibe una variable global que permite asignar identificadores únicos a los grupos.

Si el cruce que llega no tiene id, se le asigna una nueva, de tal forma que si la coordenada nueva está a una distancia menor que R de la coordenada base de un cruce, se le asigna una idéntica ya que no debe haber diferencia entre dichos vértices, a partir de icho momento serán un único vértice. En caso de que la distancia sea mayor, se le asignará un código de identificación.

Definir grupos
Esta función sirve para asignar grupos a los cruces en función de sus coordenadas x e y mediante el parámetro "R" cono límite para la asignación de grupos. Esta se realiza iterando sobre los cruces y aplicando la función anterior a aquellos que cumplen con ciertas condiciones de coordenadas.

Agrupar puntos
Esta función simplemente aplica la función anterior a todas las filas del dataframe cruces.

Crear cruces
Esta función crea cruces a partir de grupos de cruces, calculas las coordenadas promedio y la ubicación (latitud y longitud) promedio de los crues de cada grupo. Luego agrega los cruces al grafo de distancias y finalmente asigna las calles correspondientes a cada cruce y actualiza la información de las calles con los cruces asociados.

Sort cruces calle
Esta función únicamente ordena la lista de cruces.

Distance
Calcula la distancia euclidiana entre dos cruces utilizando sus coordenadas x e y.

Get aristas
Esta función calcula y devuelve las aristas del grafo entre los cruces dados.

Unir cruce
Esta función itera sobre cada calle, calcula las aristas correspondientes y las agrega al par de grafos (el de distancias y el de velocidades).

Create graphs
Esta función procesa los datos de cruces y direcciones, los cruces se ordenan según sus coordenadas x y se asignan grupos a los cruces. Se crean objetos de calle utilizando información

única de códigos, clases y nombres de calles. Se crea un grafo de distancias y se inicializa como un objeto de la clase Grafo. Se obtiene el grado de velocidades y se unen los cruces, asignando aristas a los grafos mencionados. Finalmente se mide y muestra el tiempo total que ha tardado la ejecución de la función y se devuelve una tupla con los grafos de distancias y velocidades, así como el diccionario de calles, el de nombre de calles y los dataframes originales.

Aplicación

En esta sección se va a comentar ligeramente la aplicación que va a permitir el muestreo del gps, así como un soporte sólido que permita interactuar con el mismo. No se explica en detalle debido a que no se considera relevante para la comprensión del funcionamiento en detalle del gps. Para esta sección también se hace uso de un código html contenido en html_text.py, el cual ni siquiera se va a estudiar debido a que es simple html.

La aplicación se ha creado con flask y permite al usuario introducir dos calles y un número para cada calle. Visualmente se ve un mapa de Madrid, dos ranuras para introducir las calles y un botón para enviar la respuesta, cabe destacar que el mapa es interactivo, es decir, uno se puede desplazar haciendo uso del ratón. Cuando se introduce una calle, la propia aplicación te muestra opciones que pueden estar relacionadas con la calle que se busca.

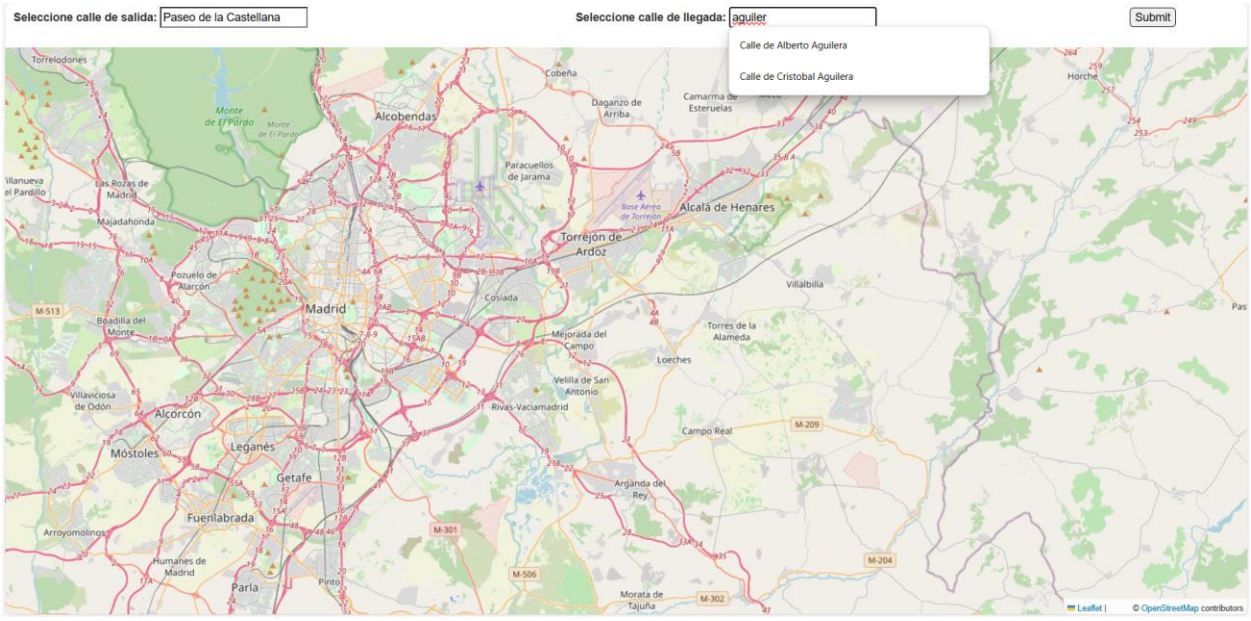


Figura : vista inicial de aplicación, autoría propia.

Una vez que se han enviado las calles, estas son verificadas de tal forma que existan en la base de datos, en caso de que sean correctas, se muestran dos nuevas ranuras para escribir el número de la calle.

Finalmente, cuando se verifica que los números son correctos, el mapa se vuelve a cargar con 6 añadidos:

- Dos banderas, una simboliza el inicio otra el final.

- Un trayecto azul que simboliza el trayecto de menor duración temporal.
- Un trayecto verde que simboliza el trayecto con menor distancia (calculada en el apartado de funciones).
- Una tabla de información correspondiente con información sobre el trayecto de menor distancia, con los giros a realizar, el tiempo y la distancia recorrida.
- Una tabla de información correspondiente con información sobre el trayecto de menor duración, con los giros a realizar, el tiempo y la distancia recorrida.

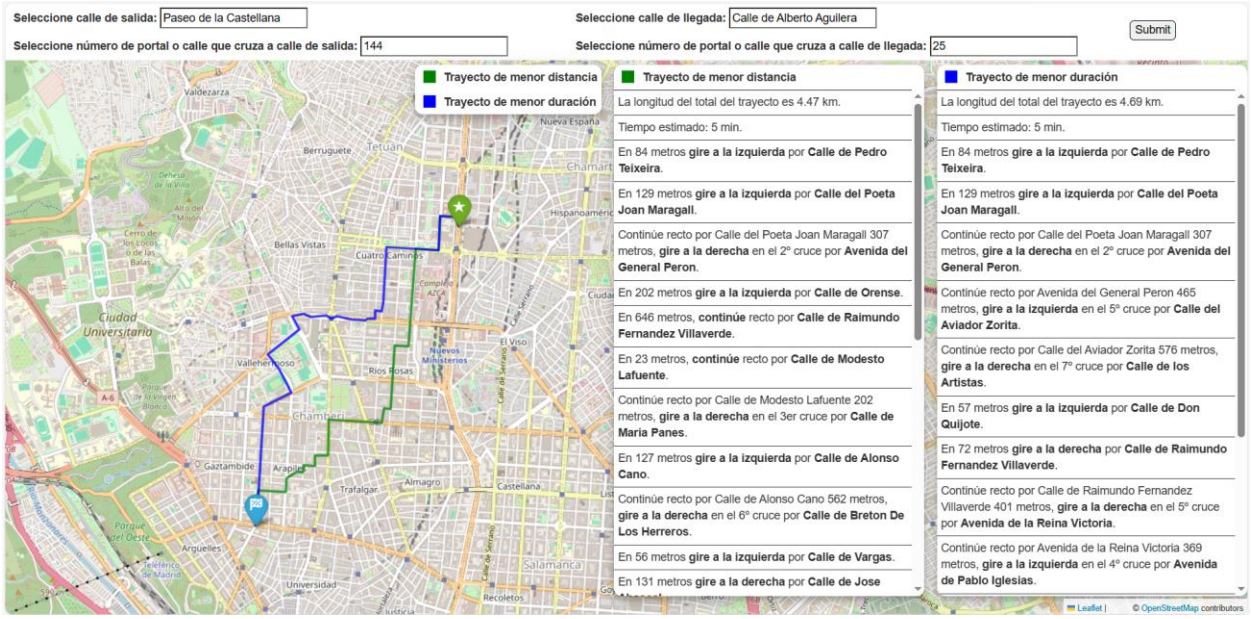


Figura : ejemplo de funcionamiento del mapa, autoría propia.